

Лекция 1. Асимптотика алгоритмов

1. Асимптотика алгоритмов

1.1 Зачем нужна асимптотика

Время работы алгоритма зависит от входа, поэтому его удобно описывать не точной формулой, а **скоростью роста** при $n \rightarrow \infty$, где n — размер входа. Это позволяет сравнивать алгоритмы независимо от конкретного железа и константных множителей.

1.2 О-нотация

Определение1 (О-большое)

Функция $f(n) = O(g(n))$, если существуют константы $c > 0$ и n_0 такие, что для всех $n \geq n_0$ выполняется

$$f(n) \leq c \cdot g(n).$$

Иными словами, $g(n)$ — асимптотическая верхняя оценка для $f(n)$.

Определение2 (Омега-большое)

Функция $f(n) = \Omega(g(n))$, если существуют константы $c > 0$ и n_0 такие, что для всех $n \geq n_0$ выполняется $f(n) \geq c \cdot g(n)$ — то есть $g(n)$ является асимптотической нижней оценкой.

Определение3 (Тета-большое)

$f(n) = \Theta(g(n))$, если одновременно $f(n) = O(g(n))$ и $f(n) = \Omega(g(n))$ — то есть $g(n)$ оценивает $f(n)$ и сверху, и снизу с точностью до константы.

Пример 1 — линейный поиск

Перебор всех n элементов массива в худшем случае делает n сравнений, значит время работы — $O(n)$.

Пример 2 — бинарный поиск

На каждом шаге бинарного поиска область поиска уменьшается вдвое, поэтому число шагов не превышает $\log_2 n + 1$ — время работы $O(\log n)$.

1.3 Худший, средний и лучший случаи

Асимптотика без уточнения обычно означает **худший случай** (worst case) — верхнюю границу для любого входа заданного размера. Средний случай (average case) требует предположений о распределении входов, а лучший случай (best case) редко даёт полезную информацию о надёжности алгоритма.

1.4 Иерархия классов сложности

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(2^n)$$

Далее в курсе эти обозначения будут использоваться без отдельных напоминаний — при анализе каждого нового алгоритма нужно уметь быстро прикинуть, к какому из этих классов он относится.